IT UNIVERSITY OF COPENHAGEN

Functional Programming

Patrick Bahr

Side Effects & Sequences

Based on original slides by Michael R. Hansen

Last week

- The memory model of F#
- Tail-recursive functions
  (aka. iterative functions)
  - Tail-recursion with accumulators
  - Tail-recursion with continuations

This week

- Side effects
  - Order of evaluation
  - Lazy evaluation
- Sequences
- Sequence expressions

Part I

Side Effects

Fibonacci numbers

```
let rec fib x =
  match x with
   | 0 -> 0
   | 1 -> 1
   | x -> fib (x - 1) +
          fib (x - 2)
```

Fibonacci numbers

```
let rec fib x =
  match x with
   | 0 -> 0
   | 1 -> 1
   | x -> fib (x - 1) +
          fib (x - 2)
```

- Not tail-recursive
- **Worse still**: $O(2^n)$ run-time
- **Reason**: same fibonacci number is recomputed many times

Fibonacci with accumulators

```
let rec fib x =
  match x with
   | 0 -> 0
   | 1 -> 1
   | x -> fib (x - 1) +
          fib (x - 2)
```

```
let fibAcc x =
  let rec aux x acc1 acc2 =
    match x with
     | 0 -> acc1
     | x -> aux (x - 1) acc2
                (acc1 + acc2)
  aux x 0 1
```

Fibonacci with accumulators

```
let rec fib x =
  match x with
  | 0 -> 0
  | 1 -> 1
  | x -> fib (x - 1) +
         fib (x - 2)
```

```
let fibAcc x =
  let rec aux x acc1 acc2 =
    match x with
    | 0 -> acc1
    | x -> aux (x - 1) acc2
               (acc1 + acc2)
  aux x 0 1
```

- `fibAcc` *is* tail-recursive
- Uses **two accumulators** to remember the previous two fibonacci numbers
- **Result**: O(n) run-time

Fibonacci with accumulators

```
let rec fib x =
  match x with
  | 0 -> 0
  | 1 -> 1
  | x -> fib (x - 1) +
         fib (x - 2)
```

```
let fibAcc x =
  let rec aux x acc1 acc2 =
    match x with
    | 0 -> acc1
    | x -> aux (x - 1) acc2
               (acc1 + acc2)
  aux x 0 1
```

- `fibAcc` *is* tail-recursive
- Uses **two accumulators** to remember the previous two fibonacci numbers
- **Result**: O(n) run-time

Memoisation

- This kind of problem can be solved elegantly using a more general technique: **Memoisation**
- Memoisation **caches** previously computed results.
- Instead of recomputing, we **look up** previous results that we cached.

Fibonacci and memoisation

```
let fibMemoize (x : int) : int =
  let mutable cache = Map.ofList [(0, 0); (1, 1)]

  let memoize (f : int -> int) (x : int) : int =
   match Map.tryFind x cache with
    | Some v -> v
    | None   ->
      let result = f x
      cache <- Map.add x result cache
      result

  let rec fib x = memoize fib (x - 1)
                + memoize fib (x - 2)
  fib x
```

Fibonacci and memoisation

```
let fibMemoize (x : int) : int =
  let mutable cache = Map.ofList [(0, 0); (1, 1)]

  let memoize (f : int -> int) (x : int) : int =
   match Map.tryFind x cache with
    | Some v -> v
    | None   ->
      let result = f x
      cache <- Map.add x result cache
      result

  let rec fib x = memoize fib (x - 1)
                + memoize fib (x - 2)
  fib x
```

Mutable variable to cache results

Fibonacci and memoisation

```
let fibMemoize (x : int) : int =
  let mutable cache = Map.ofList [(0, 0); (1, 1)]

  let memoize (f : int -> int) (x : int) : int =
   match Map.tryFind x cache with
    | Some v -> v
    | None   ->
      let result = f x
      cache <- Map.add x result cache
      result

  let rec fib x = memoize fib (x - 1)
                + memoize fib (x - 2)
  fib x
```

Mutable variable to cache results

computes f x using *memoisation*

Fibonacci and memoisation

```
let fibMemoize (x : int) : int =
  let mutable cache = Map.ofList [(0, 0); (1, 1)]

  let memoize (f : int -> int) (x : int) : int =
   match Map.tryFind x cache with
    | Some v -> v
    | None   ->
      let result = f x
      cache <- Map.add x result cache
      result

  let rec fib x = memoize fib (x - 1)
                + memoize fib (x - 2)
  fib x
```

Mutable variable to cache results

computes f x using *memoisation*

Implement fib using memoization (cases for 0 & 1 stored in cache!)

Fibonacci and memoisation

```
let fibMemoize (x : int) : int =
  let mutable cache = Map.ofList [(0, 0); (1, 1)]

  let memoize (f : int -> int) (x : int) : int =
   match Map.tryFind x cache with
    | Some v -> v
    | None   ->
      let result = f x
      cache <- Map.add x result cache
      result

  let rec fib x = memoize fib (x - 1)
                + memoize fib (x - 2)
  fib x
```

Mutable variable to cache results

computes f x using *memoisation*

Implement fib using memoization (cases for 0 & 1 stored in cache!)

NB: Memoization is overkill for this particular example, but very useful for more complex problems.

Side Effects

- Reading from and writing to the cache are **side effects** of `fibMemoize`
- Side effects are all those **observable effects** of a function other than returning a value.
- A function without side effects is called **pure**.
- Pure functions always **return the same result** if given the same arguments.
- To understand what **impure** functions do, we must understand **when** evaluation happens.

Evaluation Order

Example

```
> (1 + 2) + (3 + 4);;
val it : int = 10
```

$$(1 + 2) + (3 + 4)$$

- Expressions are evaluated **left to right**
- In `(1 + 2) + (3 + 4)` both sides of the + are pure (= *no* side effects), so the order of evaluation doesn't matter for the result.

Evaluation Order

Example

```
> (1 + 2) + (3 + 4);;
val it : int = 10
```

$$(1 + 2) + (3 + 4) \rightsquigarrow 3 + (3 + 4)$$

- Expressions are evaluated **left to right**
- In `(1 + 2) + (3 + 4)` both sides of the + are pure (= *no* side effects), so the order of evaluation doesn't matter for the result.

Evaluation Order

Example

```
> (1 + 2) + (3 + 4);;
val it : int = 10
```

$$
\begin{aligned}
& (1 + 2) + (3 + 4) \\
\rightsquigarrow\ & 3 + (3 + 4) \\
\rightsquigarrow\ & 3 + 7
\end{aligned}
$$

- Expressions are evaluated **left to right**
- In `(1 + 2) + (3 + 4)` both sides of the + are pure (= *no* side effects), so the order of evaluation doesn't matter for the result.

Evaluation Order

Example

```
> (1 + 2) + (3 + 4);;
val it : int = 10
```

$$
\begin{aligned}
& (1 + 2) + (3 + 4) \\
\rightsquigarrow\ & 3 + (3 + 4) \\
\rightsquigarrow\ & 3 + 7 \\
\rightsquigarrow\ & 10
\end{aligned}
$$

- Expressions are evaluated **left to right**
- In `(1 + 2) + (3 + 4)` both sides of the + are pure (= *no* side effects), so the order of evaluation doesn't matter for the result.

Evaluation Order

Example

```
> let three () =
    printfn "three"
    3 ;;
val three: unit -> int
```

Evaluation Order

Example

```
> let three () =
    printfn "three"
    3 ;;
val three: unit -> int


> let seven () =
    printfn "seven"
    7 ;;
val seven: unit -> int
```

Evaluation Order

Example

```
> let three () =
    printfn "three"
    3 ;;
val three: unit -> int


> let seven () =
    printfn "seven"
    7 ;;
val seven: unit -> int


> three () + seven () ;;
three
seven
val it: int = 10
```

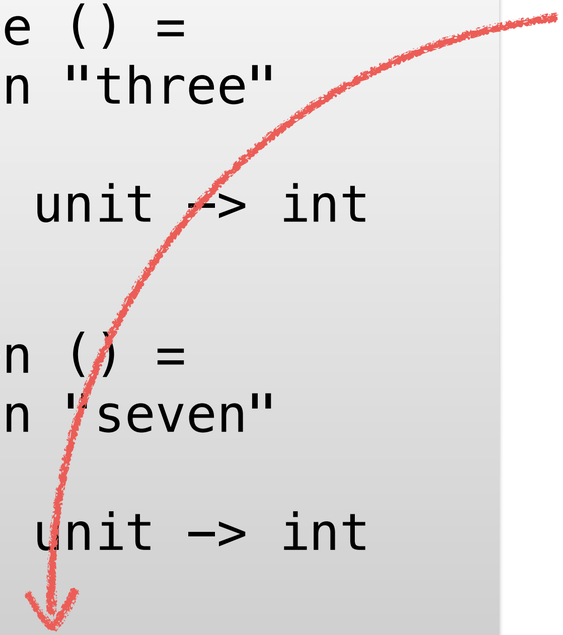
Evaluation Order

Example

```
> let three () =
    printfn "three"
    3 ;;
val three: unit -> int

> let seven () =
    printfn "seven"
    7 ;;
val seven: unit -> int

> three () + seven () ;;
three
seven
val it: int = 10
```



- Here the two arguments of + are **impure**!
- Each argument has the side effect of **printing** to the console.
- Thus, the left-to-right evaluation order becomes observable for us.
- NB: Side effects of functions are only visible when they are called

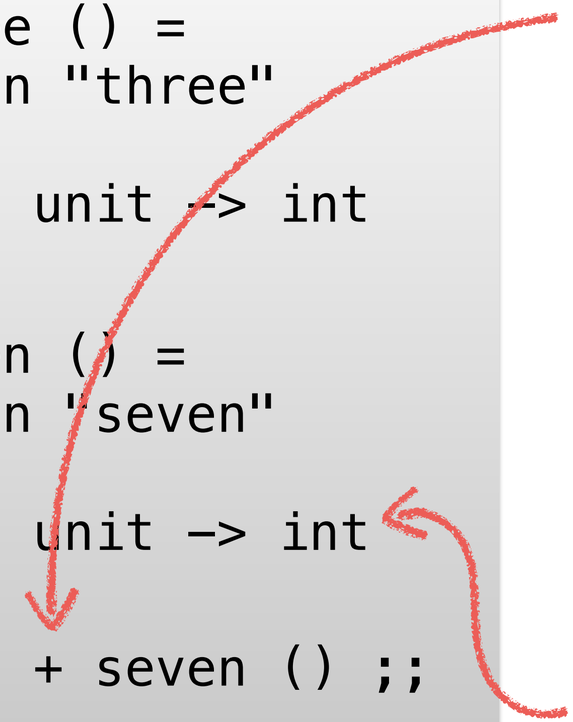
Evaluation Order

Example

```
> let three () =
    printfn "three"
    3 ;;
val three: unit -> int

> let seven () =
    printfn "seven"
    7 ;;
val seven: unit -> int

> three () + seven () ;;
three
seven
val it: int = 10
```



- Here the two arguments of + are **impure**!
- Each argument has the side effect of **printing** to the console.
- Thus, the left-to-right evaluation order becomes observable for us.
- NB: Side effects of functions are only visible when they are called

Quiz

```
let abc x =
    let foo () =
        printfn "A"
        x
    let bar () =
        printfn "B"
        foo ()
    let foobar () =
        let r = bar ()
        printfn "C"
        r
    foobar ()
```

Quiz

```
let abc x =
    let foo () =
        printfn "A"
        x
    let bar () =
        printfn "B"
        foo ()
    let foobar () =
        let r = bar ()
        printfn "C"
        r
    foobar ()
```

The following will evaluate to the value 42

```
> abc 42;;
```

It will also print the strings “A”, “B”, and “C”. But in which order?

Quiz

```
let abc x =
  let foo =
      printfn "A"
      x
  let bar = fun () ->
      printfn "B"
      foo
  let foobar () =
      let r = bar ()
      printfn "C"
      r
  foobar ()
```

How about this one?

> abc 42;;

It will also print the strings "A", "B", and "C". But in which order?

Side Effects & Evaluation Order

Example

```
> let mutable x = 5 ;;
val mutable x: int = 5

> let number () =
    x <- x + 1
    x ;;
val number: unit -> int
```

Side Effects & Evaluation Order

Example

```
> let mutable x = 5 ;;
val mutable x: int = 5

> let number () =
    x <- x + 1
    x ;;
val number: unit -> int


> number () - number () ;;
val it: int = -1
```

Side Effects & Evaluation Order

Example

```
> let mutable x = 5 ;;
val mutable x: int = 5

> let number () =
    x <- x + 1
    x ;;
val number: unit -> int


> number () - number () ;;
val it: int = -1
```

- `number` is **impure**
- In particular, `number` returns **different results** on repeated calls
- If the evaluation order were right-to-left, we would get a different result.
- **Q:** What happens if we remove () as the argument to `number`?

Lazy Evaluation

Lazy evaluation
(or *delayed evaluation*)

=

Computation is delayed
until its result is *needed*.

Lazy Evaluation

Lazy evaluation
(or *delayed evaluation*)

=

Computation is delayed
until its result is *needed*.

By default F# does *not* use lazy evaluation.

Lazy Evaluation

Lazy evaluation (or *delayed evaluation*) = Computation is delayed until its result is *needed*.

By default F# does *not* use lazy evaluation.

Example:

```
> let f x =
      let y = (x * x, x + x)
      fst y;;
> f 5;;
val it: int = 25
```

Both x * x and x + x are evaluated, even though x + x is not needed.

Lazy Evaluation

Lazy evaluation
(or *delayed evaluation*)

=

Computation is delayed until its result is *needed*.

By default F# does *not* use lazy evaluation.

Example:

```
> let f x =
      let y = (x * x, x + x)
      fst y;;
> f 5;;
val it: int = 25
```

Both x * x and x + x are evaluated, even though x + x is not needed.

Lazy Evaluation

Lazy evaluation
(or *delayed evaluation*)

=

Computation is delayed
until its result is *needed*.

Lazy Evaluation

Lazy evaluation
(or *delayed evaluation*) = Computation is delayed until its result is *needed*.

- Some languages are lazy by default (e.g. Haskell)
- We can *occasionally* use lazy evaluation in F#
- But: Lazy evaluation must be used with care since delaying evaluation is costly (why?)
- Elements of infinite sequences are lazily evaluated.

Delayed computations

The computation of the value of *e* can be delayed by ”packing” it into a function (called a closure):

```
fun () -> e
```

Delayed computations

The computation of the value of *e* can be delayed by ”packing” it into a function (called a closure):

```
fun () -> e
```

Example

```
> fun () -> 3 + 4;;
val it : unit -> int

> it ();;
val it : int = 7
```

The expression **3 + 4** is not evaluated until the function is called

Example

We can make it visible when computations are performed by the use of side effects:

```
let idWithPrint (i : int) : int =
  printfn "%d" i
  i
```

The above function takes an integer, prints it out, and finally returns it again

Examples

```
> idWithPrint 3;;
3
val it : int = 3


> (idWithPrint 3) + 1;;
3
val it : int = 4
```

Example

We can make it visible when computations are performed by the use of side effects:

```
let idWithPrint (i : int) : int =
  printfn "%d" i
  i
```

The above function takes an integer, prints it out, and finally returns it again

Examples

```
> fun () ->
    (idWithPrint 3) +
      (idWithPrint 4);;
val it : unit -> int


> it ();;
3
4
val it : int = 7
```

Example

sible when computations are
the use of side effects:

Note: Nothing is printed yet!

```
let idWithPrint (i : int) : int =
  printfn "%d" i
  i
```

The above function takes an integer, prints it out, and finally returns it again

Examples

```
> fun () ->
    (idWithPrint 3) +
      (idWithPrint 4);;
val it : unit -> int


> it ();;
3
4
val it : int = 7
```

Example

sible when computations are
the use of side effects:

Note: Nothing is printed yet!

```
let i                    int =
  pri
  i
```

Once we apply the function, the expression in it is evaluated

The above function takes an integer, prints it out, and finally returns it again

Examples

```
> fun () ->
    (idWithPrint 3) +
      (idWithPrint 4);;
val it : unit -> int


> it ();;
3
4
val it : int = 7
```

Example

sible when computations are
the use of side effects:

Note: Nothing is printed yet!

```
let i                    nt =
  pri
  i
```

Once we apply the function, the expression in it is evaluated

The above function takes
an i
and

Arguments are evaluated left to right.

Examples

```
> fun () ->
    (idWithPrint 3) +
      (idWithPrint 4);;
val it : unit -> int


> it ();;
3
4
val it : int = 7
```

Eager Evaluation of Pairs

Let's reconsider the previous example of a pair where we only use the first component:

Example

```
> let f x =
      let y = (idWithPrint (x * x), idWithPrint (x + x))
      fst y;;
val f: x: int -> int

> f 5;;
25
10
val it: int = 25
```

Eager Evaluation of Pairs

Let's reconsider the previous example of a pair where we only use the first component:

Example

```
> let f x =
      let y = (idWithPrint (x * x), idWithPrint (x + x))
      fst y;;
val f: x: int -> int

> f 5;;
25
10
val it: int = 25
```

We can see that **both** components of the pair are evaluated, even though we only use the first one.

Summary

- A side effects is an **observable effects** of a function other than returning a value, e.g.
  - printing to console, reading/writing from file
  - reading/writing a mutable variable or array
- Understanding **order of evaluation** becomes important for **impure** functions (i.e. functions with side effects).
- Sometimes we can **encapsulate side effects** to obtain more efficient implementations, e.g. **memoisation**

Questions?

Part II

Sequences

Sequences

- We are already very familiar with **lists** (type `list<'a>`)
- **Sequences** (type `seq<'a>`) behave a lot like lists, but
  - sequences can be **lazy**
    (i.e. their elements may be computed on demand)
  - sequences can be **infinite**
    (and that can be surprisingly useful!)
- We'll see shortly why lazy and/or infinite sequences might be useful

Constructing sequences

Recall **list literals**:

```
>[1; 2; 3];;
val it : int list = [1; 2; 3]
```

Constructing sequences

Recall **list literals**:

```
>[1; 2; 3];;
val it : int list = [1; 2; 3]
```

Sequences can be constructed with **sequence literals** similarly to lists literals:

```
> seq [1; 2; 3];;
val it : seq<int> = [1; 2; 3]
```

Constructing sequences

We can use functions to create *finite* sequences:

```
Seq.init : int -> (int -> 'a) -> seq<'a>
```

Constructing sequences

We can use functions to create *finite* sequences:

```
Seq.init : int -> (int -> 'a) -> seq<'a>
```

This is (obviously) not how Seq.init is implemented, but rather an intuitive description of its behaviour.

Constructing sequences

We can use functions to create *finite* sequences:

```
Seq.init : int -> (int -> 'a) -> seq<'a>
```

```
> Seq.init 4 (fun n -> n*n);;
val it : seq<int> = seq [0; 1; 4; 9]
```

Constructing sequences

… and *infinite* sequences:

```
Seq.initInfinite : (int -> 'a) -> seq<'a>
Seq.initInfinite f = seq [f 0; f 1; …]
```

Constructing sequences

… and *infinite* sequences:

```
Seq.initInfinite : (int -> 'a) -> seq<'a>
Seq.initInfinite f = seq [f 0; f 1; …]
```

```
> Seq.initInfinite (fun n -> n*n);;
val it : seq<int> = seq [0; 1; 4; 9; ...]
```

Constructing sequences

… and *infinite* sequences:

```
Seq.initInfinite : (int -> 'a) -> seq<'a>
Seq.initInfinite f = seq [f 0; f 1; …]
```

```
> Seq.initInfinite (fun n -> n*n);;
val it : seq<int> = seq [0; 1; 4; 9; ...]
```

How is this possible? How can we work with infinite objects?

Sequences

A sequence is a (possibly infinite) ordered collection of values.

```
> let nat = Seq.initInfinite (fun x -> x);;      // seq [0; 1; 2; 3; …]
val nat : seq<int>

> let pos = Seq.initInfinite (fun x -> x + 1);; // seq [1; 2; 3; 4; …]
val pos : seq<int>
```

```
> Seq.item 4 nat;;
val it : int = 4
```

```
> Seq.item 4 pos;;
val it : int = 5
```

Sequences

Items from sequences are **computed on demand**

```
> let nat = Seq.initInfinite idWithPrint;;
val nat : seq<int>                                  // seq [0; 1; 2; 3; …]

> Seq.item 4 nat;;
4
val it : int = 4

> Seq.item 5 nat;;
5
val it : int = 5

> Seq.item 4 nat;;
4
val it : int = 4
```

Sequences

Items from sequences are **computed on demand**

```
> let nat = Seq.initInfinite idWithPrint;;
val nat : seq<int>                          // seq [0; 1; 2; 3; …]

> Seq.item 4 nat;;
4
val it : int = 4

> Seq.item 5 nat;;
5
val it : int = 5

> Seq.item 4 nat;;
4
val it : int = 4
```

```
let idWithPrint (i : int) : int =
  printfn "%d" i
  i
```

Sequences

Items from sequences are **computed on demand**

```
> let nat = Seq.initInfinite idWithPrint;;
val nat : seq<int>                                  // seq [0; 1; 2; 3; …]

> Seq.item 4 nat;;
4
val it : int = 4

> Seq.item 5 nat;;
5
val it : int = 5

> Seq.item 4 nat;;
4
val it : int = 4
```

```
let idWithPrint (i : int) : int =
  printfn "%d" i
  i
```

We have to **recompute** the element of the sequence

Caching

Recomputation can be avoided by using a *cache*

```
Seq.cache : seq<'a> -> seq<'a>

> let nat = Seq.initInfinite idWithPrint;;
val nat : seq<int>

> let natCache = Seq.cache nat;;
val natCache : seq<int>
```

Caching

Recomputation can be avoided by using a *cache*

```
Seq.cache : seq<'a> -> seq<'a>

> let nat = Seq.initInfinite idWithPrint;;
val nat : seq<int>

> let natCache = Seq.cache nat;;
val natCache : seq<int>
```

Example

```
> Seq.item 2 natCache;;
0
1
2
val it : int = 2
```

Caching

Recomputation can be avoided by using a *cache*

```
Seq.cache : seq<'a> -> seq<'a>

> let nat = Seq.initInfinite idWithPrint;;
val nat : seq<int>

> let natCache = Seq.cache nat;;
val natCache : seq<int>
```

Example

```
> Seq.item 2 natCache;;
0
1
2
val it : int = 2

> Seq.item 2 natCache;;
val it : int = 2
```

Caching

Recomputation can be avoided by using a *cache*

```
Seq.cache : seq<'a> -> seq<'a>

> let nat = Seq.initInfinite idWithPrint;;
val nat : seq<int>

> let natCache = Seq.cache nat;;
val natCache : seq<int>
```

Example

```
> Seq.item 2 natCache;;
0
1
2
val it : int = 2

> Seq.item 2 natCache;;
val it : int = 2

> Seq.item 4 natCache;;
3
4
val it : int = 4
```

Caching

Recomputation can be avoided by using a *cache*

```
Seq.cache : seq<'a> -> seq<'a>

> let nat = Seq.initInfinite idWithPrint;;
val nat : seq<int>

> let natCache = Seq.cache nat;;
val natCache : seq<int>
```

- a cached sequence has first n elements cached
- when looking up element i < n, then cached value is returned (no computation required)
- otherwise (i ≥ n) the elements n, … , i are computed and stored in the cache

Example

```
> Seq.item 2 natCache;;
0
1
2
val it : int = 2

> Seq.item 2 natCache;;
val it : int = 2

> Seq.item 4 natCache;;
3
4
val it : int = 4
```

Filtering

A sequence of even natural numbers is obtained by filtering

```
> let even = Seq.filter (fun n -> n%2=0) nat;;
val even : seq<int>


> Seq.toList(Seq.take 4 even);;
0
1
2
3
4
5
6
val it : int list = [0; 2; 4; 6]
```

Filtering

A sequence of even natural numbers is obtained by filtering

```
> let even = Seq.filter (fun n -> n%2=0) nat;;
val even : seq<int>

> Seq.toList(Seq.take 4 even);;
0
1
2
3
4
5
6
val it : int list = [0; 2; 4; 6]
```

Calculating the first four even numbers requires computing the first seven natural numbers

Filtering

A sequence of even natural numbers is obtained by filtering

```
> let even = Seq.filter (fun n -> n%2=0) nat;;
val even : seq<int>

> Seq.toList(Seq.take 4 even);;
0
1
2
3
4
5
6
val it : int list = [0; 2; 4; 6]
```

Calculating the first four even numbers requires computing the first seven natural numbers

By using an infinite sequence, we don't have to worry about how many natural numbers we need to check!

Delay

Let’s try to define `seq [0; 1; 2; 3; …]` by recursion:

```
let rec from n =
    Seq.append (seq [n]) (from (n+1))
```

Delay

Let’s try to define `seq [0; 1; 2; 3; …]` by recursion:

```
let rec from n =
    Seq.append (seq [n]) (from (n+1))
```

Idea

`from n = seq [n; n+1;…]`

Delay

Let's try to define `seq [0; 1; 2; 3; …]` by recursion:

```
let rec from n =
    Seq.append (seq [n]) (from (n+1))
```

`from 0` will loop forever!

```
> from 0;;
  Stack overflow
```

Delay

Let's try to define `seq [0; 1; 2; 3; …]` by recursion:

```
let rec from n =
    Seq.append (seq [n]) (from (n+1))
```

`from 0` will loop forever!

```
> from 0;;
  Stack overflow
```

We have to delay the recursive call to `from`

```
Seq.delay : (unit -> seq<'a>) -> seq<'a>
```

```
let rec from n =
    Seq.append (seq [n])
      (Seq.delay (fun () -> from (n+1)))
```

Delay

Let's try to define `seq [0; 1; 2; 3; …]` by recursion:

```
let rec from n =
    Seq.append (seq [n]) (from (n+1))
```

```
from 0 will loop forever!

> from 0;;
  Stack overflow
```

We have to delay the recursive call to `from`

```
Seq.delay : (unit -> seq<'a>) -> seq<'a>
```

```
let rec from n =
    Seq.append (seq [n])
      (Seq.delay (fun () -> from (n+1)))
```

```
> let nat = from 0;;
val nat : seq<int>

> Seq.item 4 nat;;
val it : int = 4
```

Part III

Sequence Expressions

Sieve of Eratosthenes

Computing the sequence of prime numbers using the following simple procedure:

- Start with the sequence 2, 3, 4, 5, 6, …

Sieve of Eratosthenes

Computing the sequence of prime numbers using the following simple procedure:

2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, …

- Start with the sequence 2, 3, 4, 5, 6, …

Sieve of Eratosthenes

Computing the sequence of prime numbers
using the following simple procedure:

2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, …

- Start with the sequence 2, 3, 4, 5, 6, …
  select the head (2) and remove multiples of 2 from the sequence

Sieve of Eratosthenes

Computing the sequence of prime numbers
using the following simple procedure:

- Start with the sequence 2, 3, 4, 5, 6, …
  select the head (2) and remove multiples of 2 from the sequence

Sieve of Eratosthenes

Computing the sequence of prime numbers
using the following simple procedure:

- Start with the sequence 2, 3, 4, 5, 6, …
  select the head (2) and remove multiples of 2 from the sequence
- The sequence is now 3, 5, 7, 9, 11, …

Sieve of Eratosthenes

Computing the sequence of prime numbers
using the following simple procedure:

- Start with the sequence 2, 3, 4, 5, 6, …
  select the head (2) and remove multiples of 2 from the sequence
- The sequence is now 3, 5, 7, 9, 11, …
  select the head (3) and remove multiples of 3 from the sequence

Sieve of Eratosthenes

Computing the sequence of prime numbers
using the following simple procedure:

- Start with the sequence 2, 3, 4, 5, 6, …
  select the head (2) and remove multiples of 2 from the sequence
- The sequence is now 3, 5, 7, 9, 11, …
  select the head (3) and remove multiples of 3 from the sequence

Sieve of Eratosthenes

Computing the sequence of prime numbers
using the following simple procedure:

- Start with the sequence 2, 3, 4, 5, 6, …
  select the head (2) and remove multiples of 2 from the sequence
- The sequence is now 3, 5, 7, 9, 11, …
  select the head (3) and remove multiples of 3 from the sequence
- The sequence is now 5, 7, 11, 13, 17

Sieve of Eratosthenes

Computing the sequence of prime numbers
using the following simple procedure:

- Start with the sequence 2, 3, 4, 5, 6, …
  select the head (2) and remove multiples of 2 from the sequence
- The sequence is now 3, 5, 7, 9, 11, …
  select the head (3) and remove multiples of 3 from the sequence
- The sequence is now 5, 7, 11, 13, 17
  select the head (5) and remove multiples of 5 from the sequence

Sieve of Eratosthenes

Computing the sequence of prime numbers
using the following simple procedure:

- Start with the sequence 2, 3, 4, 5, 6, …
  select the head (2) and remove multiples of 2 from the sequence
- The sequence is now 3, 5, 7, 9, 11, …
  select the head (3) and remove multiples of 3 from the sequence
- The sequence is now 5, 7, 11, 13, 17
  select the head (5) and remove multiples of 5 from the sequence
- …

Sieve of Eratosthenes

Remove multiples of *a* from sequence *sq*:

```
let sift a sq = Seq.filter (fun n -> n % a <> 0) sq
```

Sieve of Eratosthenes

Remove multiples of *a* from sequence *sq*:

```
let sift a sq = Seq.filter (fun n -> n % a <> 0) sq
```

Select the head and remove multiples of the head from the sequence:

```
let rec sieve sq =
    Seq.delay (fun () ->
        let p = Seq.item 0 sq
        Seq.append
            (Seq.singleton p)
            (sieve(sift p (Seq.skip 1 sq))))
```

Sieve of Eratosthenes

Remove multiples of *a* from sequence *sq*:

```
let sift a sq = Seq.filter (fun n -> n % a <> 0) sq
```

Select the head and remove multiples of the head from the sequence:

```
let rec sieve sq =
    Seq.delay (fun () ->
        let p = Seq.item 0 sq
        Seq.append
            (Seq.singleton p)
            (sieve(sift p (Seq.skip 1 sq))))
```

Seq.delay is needed to avoid inifinite recursion

Putting it all together

```
let numFrom2 = Seq.initInfinite (fun n -> n+2)    // seq [2; 3; 4; …]
let primes   = sieve numFrom2                     // seq [2; 3; 5; …]
```

Putting it all together

```
let numFrom2 = Seq.initInfinite (fun n -> n+2)    // seq [2; 3; 4; …]
let primes   = sieve numFrom2                     // seq [2; 3; 5; …]

let nthPrime n = Seq.item n primes                // n-th prime
```

Putting it all together

```
let numFrom2 = Seq.initInfinite (fun n -> n+2)   // seq [2; 3; 4; …]
let primes   = sieve numFrom2                    // seq [2; 3; 5; …]

let nthPrime n = Seq.item n primes               // n-th prime
```

```
> nthPrime 1000;;
 Real: 00:00:07.200, CPU: 00:00:07.209, GC gen0: 272, gen1: 3
 val it : int = 7927

> nthPrime 1001;;
 Real: 00:00:07.021, CPU: 00:00:07.081, GC gen0: 272, gen1: 3
 Val it : int = 7933
```

Caching the sequence of primes

Recomputation can be avoided by using a cache

```
let primesCached =
    Seq.cache primes

let nthPrime' n =
    Seq.item n primesCached
```

Caching the sequence of primes

Recomputation can be avoided by using a cache

```
let primesCached =
    Seq.cache primes

let nthPrime' n =
    Seq.item n primesCached
```

```
> nthPrime' 1000;;
 Real: 00:00:07.023, CPU: 00:00:07.056, GC gen0: 272, gen1: 2
 val it : int = 7927

> nthPrime' 1001;;
 Real: 00:00:00.021, CPU: 00:00:00.023, GC gen0: 0, gen1: 0
 Val it : int = 7933
```

Sieve of Eratosthenes

We can use **sequence expressions** to write `sieve`

```
let rec sieve sq =
    seq { let p = Seq.item 0 sq
          yield p
          yield! sieve (sift p (Seq.skip 1 sq)) }
```

- Implicitly lazy construction; `Seq.delay` is not needed
- `yield x` inserts the element `x`
- `yield! sq` inserts the sequence `sq`

Sieve of Eratosthenes

```
let rec sieve sq =
    seq { let p = Seq.item 0 sq
          yield p
          yield! sieve (sift p (Seq.skip 1 sq)) }

let sift a sq =
    seq { for n in sq do
            if n % a <> 0 then yield n }
```

- `for p in sq do` iterates over sequence `sq`
- `if b then sq` only includes `sq` if `b` is true

Sequence expressions

```
> seq {for x in 1..10 do yield x * x}

val it : seq<int> = seq [1; 4; 9; 16; …]
```

- Sequence expressions are a special case of **computation expressions**.
- We'll learn about these next week!
- This is just a preview.

A computation expression is defined like this:

```
type MySeqBuilder() =
  member this.Yield x = …
  …
let seq = MySeqBuilder()
```

Sequence expressions

```
> seq {for x in 1..10 do yield x * x}
```

The above syntax is translated as follows:

```
T(yield v)                  = Yield v

T(yield! e)                 = YieldFrom e

T(for x in e do ce)         = For(e, fun x -> T(ce))
```

Sequence expressions

```
> seq {for x in 1..10 do yield x * x}
```

The above syntax is translated as follows:

T is the function that does the translation.

```
T(yield v)                  = Yield v

T(yield! e)                 = YieldFrom e

T(for x in e do ce)         = For(e, fun x -> T(ce))
```

Sequence expressions

```
> seq {for x in 1..10 do yield x * x}
```

In addition, computation expressions
for sequences use the function `Delay`

```
T(seq {ce})      = Delay (fun () -> T (ce))
```

Sequence expressions

```
> seq {for x in 1..10 do yield x * x}
```

In addition, computation expressions for sequences use the function `Delay`

```
T(seq {ce})      = Delay (fun () -> T (ce))
```

This makes sure that sequences defined by `seq {…}` are delayed by default (⇒ can be used in recursive functions)

Sequences Everywhere

- `seq<'a>` is just an alias for `IEnumerable<'a>`
- Anything that implements `IEnumerable` is also a sequence
- strings, lists, arrays etc.

Sequences Everywhere

- `seq<'a>` is just an alias for `IEnumerable<'a>`
- Anything that implements `IEnumerable` is also a sequence
- strings, lists, arrays etc.


```
> Seq.zip [1;2;3] "abc";;

val it : seq<int * char> = seq [(1, 'a'); (2, 'b'); (3, 'c')]
```

Sequence comprehensions

Instead of `seq {...}` notation,
we can also write `seq [...]`.

```
> seq {for x in 1..10 do yield x * x}
val it: int seq = [1; 4; 9; 16; ...]

> seq [for x in 1..10 do yield x * x]
val it: int seq = [1; 2; 3; 4; 5; 6; 7; 8; 9; 10]
```

This `seq [...]` notation is also called **sequence comprehension**.

Corresponding notations exists for other collection types.

Sequence comprehensions

Sequence comprehensions have slightly **different semantics**: They **don't** implicitly insert `Seq.delay`!

```
> let rec from n = seq {yield n; yield! from (n+1)};;
val from: n: int -> int seq

> from 10;;
val it: int seq = seq [10; 11; 12; 13; ...]
```

Sequence comprehensions

Sequence comprehension have slightly different semantics: They don't implicitly insert delays!

```
> let rec from n = seq [yield n; yield! from (n+1)];;
val from: n: int -> int seq

> from 10;;
Stack overflow.
```

Collection comprehensions

Similarly to sequence comprehensions, we also have comprehension notations for other collections!

```
> [1..10]
val it: int list = [1; 2; 3; 4; 5; 6; 7; 8; 9; 10]

> [|1..10|]
val it: int array = [|1; 2; 3; 4; 5; 6; 7; 8; 9; 10|]

> seq [1..10]
val it: int seq = [1; 2; 3; 4; 5; 6; 7; 8; 9; 10]
```

Collection comprehensions

Similarly to sequence comprehensions, we also have comprehension notations for other collections!

```
> [1..2..10]
val it: int list = [1; 3; 5; 7; 9]

> [|1..2..10|]
val it: int array = [|1; 3; 5; 7; 9|]

> seq [1..2..10]
val it: int seq = [1; 3; 5; 7; 9]
```

Collection comprehensions

Similarly to sequence comprehensions, we also have comprehension notations for other collections!

We can safely loop using list comprehension

```
> [for x in 1..10 do yield x * 2]
val it: int list = [2; 4; 6; 8; 10; 12; 14; 16; 18; 20]
```

Collection comprehensions

Similarly to sequence comprehensions, we also have comprehension notations for other collections!

We can safely loop using list comprehension

```
> [for x in 1..10 do yield x * 2]
val it: int list = [2; 4; 6; 8; 10; 12; 14; 16; 18; 20]
```

or shorter

```
> [for x in 1..10 -> x * 2]
val it: int list = [2; 4; 6; 8; 10; 12; 14; 16; 18; 20]
```

Collection comprehensions

Similarly to sequence comprehensions, we also have comprehension notations for other collections!

We can also loop over another collection

```
> let lst = [1..10]
val lst: int list = [1; 2; 3; 4; 5; 6; 7; 8; 9; 10]

> [for x in lst -> x * 2]
val it: int list = [2; 4; 6; 8; 10; 12; 14; 16; 18; 20]
```

Collection comprehensions

Similarly to sequence comprehensions, we also have comprehension notations for other collections!

We can also loop over another collection even if it is of another collection type!

```
> let arr = [|1..10|]
val arr: int array = [|1; 2; 3; 4; 5; 6; 7; 8; 9; 10|]

> [for x in arr -> x * 2]
val it: int list = [2; 4; 6; 8; 10; 12; 14; 16; 18; 20]
```

Summary

- Sequences are a lot **like lists**, but **more general** (They have many of the same functions: fold, map, filter etc.)
- Sequences are **more flexible**
  - they can be **infinite**
  - elements can be **computed on-the-fly** (i.e. lazily)
- **Sequence expressions / comprehensions** allow for convenient manipulation and construction of sequences

Questions?